Interview Handbook

Table of Contents

1. Object-Oriented Programming
2. SOLID Design Principles
3. SOLID Principles
4. Java Memory Model

1

2

---

asdasda

asdasd

Object-Oriented Programming

> **OOP_Principles**
>
> # Object-Oriented Programming (OOP) Principles
>
> OOP is a programming paradigm based on the concept of **"objects"**, which can contain data (attributes) and code (methods). It aims to implement real-world entities like inheritance, hiding, and polymorphism in programming.
>
> ---
>
> ## 1. Encapsulation (Data Hiding)
>
> Encapsulation is the bundling of data and the methods that operate on that data into a single unit (class). It restricts direct access to some of an object's components.

- **Key Concept:** Use `private` variables and `public` getters/setters.
- **Why?** It protects the internal state of an object from unauthorized access and keeps the code modular.
- **Interview Tip:** If asked "Why not just use public variables?", answer: *"To maintain control over data validation and protect the object's integrity (Invariants)."*

[Image of Object-Oriented Programming Encapsulation diagram]

---

2. Inheritance

Inheritance allows a class (subclass/child) to acquire the properties and behaviors of another class (superclass/parent).

- **Key Concept:** The `extends` keyword in Java.
- **Why?** It promotes **code reusability** and establishes a "is-a" relationship.
- **Interview Tip:** Be careful with the "Diamond Problem" (multiple inheritance). Remember that Java solves this by not allowing multiple class inheritance but allowing multiple **interface** implementation.

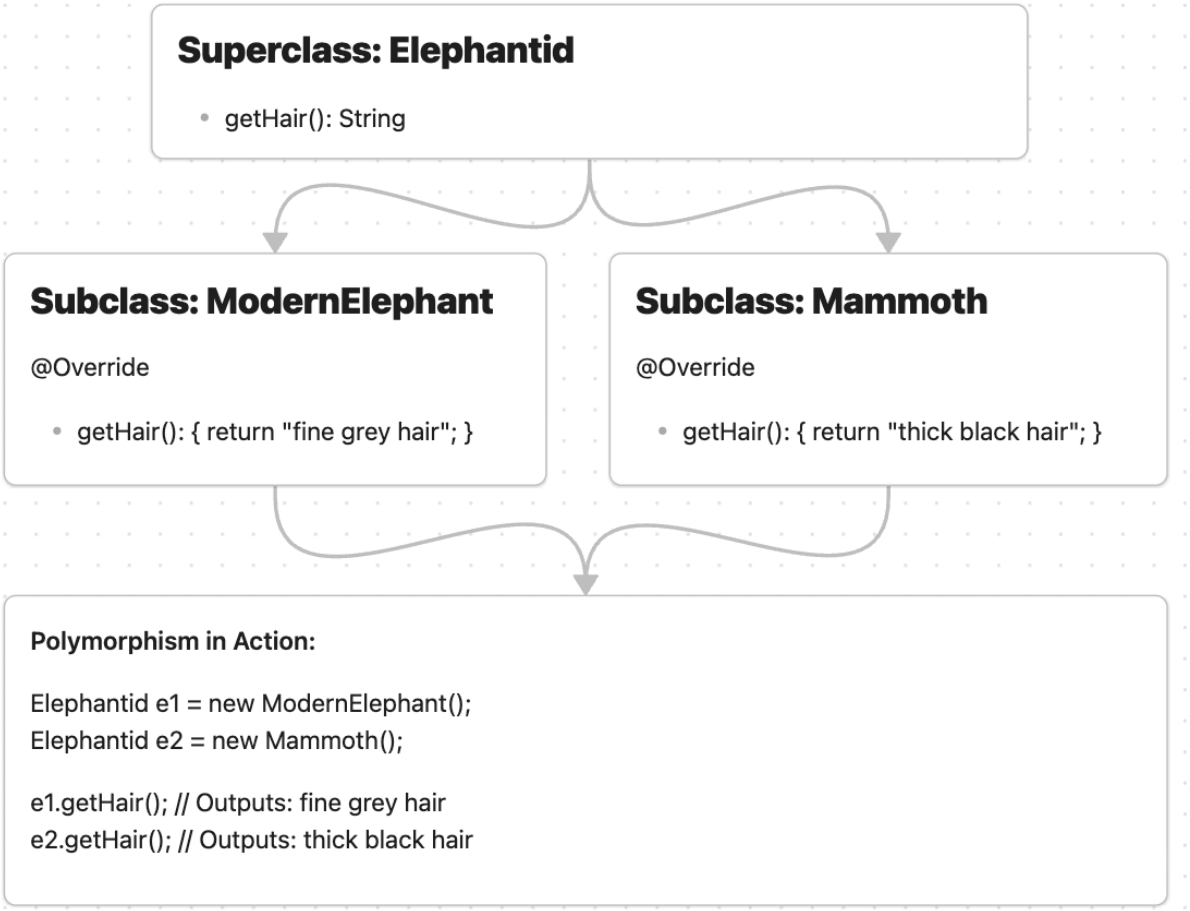
---

3. Polymorphism (Many Forms)

Polymorphism allows objects of different classes to be treated as objects of a common superclass.

- **Two Types:**
  1. **Static (Compile-time):** Method Overloading (same name, different parameters).
  2. **Dynamic (Runtime):** Method Overriding.

- **Elephantid Example:**



---

4. Abstraction

Abstraction is the process of hiding the implementation details and showing only the essential features of the object.

- **Key Concept:** `abstract` classes and `interfaces`.
- **Abstraction vs. Encapsulation:** * *Abstraction* is about **hiding complexity** (What it does).
  - *Encapsulation* is about **hiding data** (How it stores it).
- **Interview Tip:** "When to use an Interface vs. an Abstract Class?"
  - Use **Interface** for "can-do" (behavior) capabilities.
  - Use **Abstract Class** for "is-a" (identity) relationships.

Related Topics

- SOLID Principles
- Composition vs Inheritance

- Design Patterns

SOLID Design Principles

SOLID_Principles

SOLID Design Principles

SOLID is a mnemonic acronym for five design principles intended to make software designs more understandable, flexible, and maintainable. These principles are the backbone of professional software engineering.

---

1. S: Single Responsibility Principle (SRP)

> "A class should have one, and only one, reason to change."

The Core: A class should do one thing and do it well. If a class has multiple responsibilities, they become coupled, leading to fragile designs.

- **Bad:** A `User` class that handles data validation, database persistence, and email notifications.
- **Good:** Split into `User` (Model), `UserRepository` (Persistence), and `EmailService` (Communication).

---

2. O: Open/Closed Principle (OCP)

> "Software entities should be open for extension, but closed for modification."

The Core: You should be able to add new functionality without touching existing, tested code.

- **Method:** Use Interfaces and Abstraction.
- **Example:** Instead of adding an `if-else` inside a `PaymentProcessor` for every new currency, create a `PaymentStrategy` interface. New currencies just implement the interface without changing the processor.

---

3. L: Liskov Substitution Principle (LSP)

> "Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

The Core: A subclass must fulfill the "contract" of its parent. It should not withdraw or change expected behaviors.

- **Classic Violation:** The Square-Rectangle problem. If a `Square` inherits from `Rectangle` but breaks the logic of independent width/height setting, it violates LSP.
- **Interview Tip:** "Is-a" relationship is not enough; "Behavioral Compatibility" is mandatory.

---

4. I: Interface Segregation Principle (ISP)

> "Clients should not be forced to depend upon interfaces that they do not use."

The Core: Better to have many specific interfaces than one general-purpose "Fat Interface."

- **Example:** If you have an `IMachine` interface with `print()`, `scan()`, and `fax()`, a simple printer shouldn't be forced to implement `fax()`. Split them into `IPrinter`, `IScanner`, and `IFax`.

---

5. D: Dependency Inversion Principle (DIP)

> "High-level modules should not depend on low-level modules. Both should depend on abstractions."

The Core: Depend on **Interfaces**, not **Concrete Classes**. This decouples your system components.

- **Example:** A `Notification` class should not depend on `GmailService` directly. It should depend on an `IMessageService` interface. This allows you to swap Gmail for SendGrid or SMS without changing the `Notification` logic.

Related Topics

- OOP Principles
- Design Patterns
- Clean Architecture

Java Memory Management

“06_Java/Memory_Management” is not created yet. Click to create.